Rapport de projet d'introduction à la sécurité

Roy Nicolas
Lou-Ann Coquard-Morel
Théo Massenya
Mathis Hamri
Dylan-Thomas Houngbedji

Contents

1 Introduction 1

2 Fonctionnalités 2

3 Sécurité 3

4 Serveur et déploiement 4

5 Interface de l'application 5

6 Parcours utilisateur 6

1 Introduction

Application SHHH - Système de Messagerie Chiffrée

SHHH est une application mobile de messagerie instantanée conçue avec une approche de sécurité maximale, intégrant un chiffrement de bout en bout pour garantir la confidentialité des communications entre utilisateurs.

L'application est développée en **Flutter**, un framework que nous maîtrisons bien, ce qui nous a permis de réutiliser et d'adapter du code provenant de projets antérieurs.

Du point de vue de la sécurité, nous utilisons `flutter_secure_storage` pour stocker les éléments sensibles (clé privée RSA, clé privée Diffie–Hellman, secret partagé, token d'authentification, identifiants utilisateurs, etc.), tandis que les messages sont conservés dans une base de données locale **SQLite** chiffrée et déchiffrés à chaque ouverture de conversation.

`flutter_secure_storage` s'appuie sur **Android Keystore** sous Android et sur **Keychain** sous iOS, empêchant ainsi un attaquant ayant accès au système de fichiers de récupérer les clés privées ou les messages chiffrés, même en cas de root, de jailbreak ou de malware lisant les fichiers.

Cette protection ne couvre toutefois pas un malware déjà exécuté sur l'appareil, qui pourrait agir au moment où les données sont déchiffrées.

2 Fonctionnalités

La sécurité constitue le cœur de notre application:

- Chiffrement de bout en bout: Toutes les communications entre utilisateurs sont chiffrées:
  - Hachage sécurisé des données sensibles
  - Stockage sécurisé des clés de chiffrement
- Système de Messagerie
  Notre application propose plusieurs modes de communication:
  - Conversations Privées
  - Discussions entre deux utilisateurs
  - Historique de messages persistant
  - Informations messagerie classiques: date d'envoi, statut de lecture, identification des expéditeurs, etc.
  - Création et gestion de groupes de discussions
  - Partage de messages cryptés au sein du groupe
  - Envoi et réception de messages en temps réel
  - chiffrement/déchiffrement automatique
- Persistance et Stockage des Données (gestion du stockage local):
  - Base de données SQLite: Stockage persistant des messages, conversations et groupes
  - SharedPreferences: Sauvegarde des préférences utilisateur
- Protection
  Possibilité d'effacer toutes nos traces sur l'application:
  - Déconnexion
  - Fonction FULL_RESET: supprime l'ensemble des messages, conversations, clés, clés partagées, tokens, etc.

3 Sécurité

Les protocoles principaux

- Diffie-Hellman (DH) – C'est le cœur de l'échange de clés. Chaque utilisateur génère un secret (a ou b), calcule sa clé publique (A ou B), et les deux peuvent dériver la même clé partagée K sans jamais l'avoir transmise sur le réseau.
- AES-256-GCM – Le chiffrement symétrique des messages. Une fois la clé partagée obtenue via DH, elle est utilisée pour chiffrer rapidement les messages. Le mode GCM est particulièrement intéressant car il authentifie en même temps qu'il chiffre, garantissant l'intégrité du message.
- RSA – Pour les signatures numériques. Chaque utilisateur a sa paire de clés (privée/publique). La clé publique est sur le serveur, la privée reste locale. Ça permet de prouver que c'est bien toi qui as envoyé le message.
- SHA-256 – Pour le hachage. Utilisé notamment pour dériver la clé AES à partir du secret Diffie-Hellman, et pour vérifier l'intégrité des données.
  Dans notre système, SHA-256 a deux usages principaux. D'abord, elle transforme le secret Diffie-Hellman (jusqu'à 2048 bits) en clé AES-256 utilisable (256 bits). Ensuite, elle crée une empreinte des messages pour la signature RSA : plutôt que signer directement un message potentiellement volumineux, on signe son hash de 256 bits, ce qui garantit l'authenticité et l'intégrité du message.

  Nous utilisons SHA-256 car elle offre le meilleur compromis. SHA-512 serait plus lent sans gain de sécurité, MD5 et SHA-1 sont obsolètes et vulnérables, et SHA3-256, bien que sûr, reste moins répandu. SHA-256 est devenu le standard universel de l'industrie.

  SHA-256 ne chiffre pas les données, elle sert de "colle" entre techniques cryptographiques en résolvant des problèmes de compatibilité de taille tout en renforçant la sécurité du système.
- HTTPS/TLS – La base ! Tout le transport client-serveur est chiffré.

Le flux en pratique

- Inscription : Génération RSA + DH, envoi des clés publiques au serveur
- Échange de clés : Alice et Bob calculent leur clé partagée via DH (jamais transmise !)
- Envoi : Message chiffré en AES + signature RSA du ciphertext——nonce——tag
- Réception : Vérification de la signature RSA, puis déchiffrement AES

C'est un système solide qui combine le meilleur des deux mondes : la rapidité du chiffrement symétrique (AES) avec la sécurité de l'échange de clés asymétrique (DH + RSA). Le serveur ne voit jamais les clés privées ni le contenu des messages !

4 Serveur et déploiement

Nicolas Roy

Le serveur est développé en Flask et utilise SQLAlchemy. C'est un environnement que je maîtrise déjà bien, ce qui m'a permis de mettre en place le backend rapidement.

Nous exposons principalement :

- des routes REST ;
- des WebSockets pour la réception des messages en temps réel.

Nous respectons autant que possible les bonnes pratiques des API RESTful.

Les mots de passe sont hachés avec **scrypt** via la bibliothèque `werkzeug.security`.

La base de données utilise **MariaDB**, là encore par choix de continuité avec des technologies que je connaissais déjà bien.

Pour le déploiement, nous utilisons :

- **GitLab CI/CD** pour builder l'image du conteneur backend à chaque push sur la branche `main` ;
- **GitHub Actions** pour déployer automatiquement les conteneurs sur le serveur.

Les tokens et mots de passe sont gérés via les **GitHub Secrets**, ce qui permet de les déployer de manière sécurisée.

Chaque service est conteneurisé (backend, MariaDB, Nginx) et déployé automatiquement. Ce pipeline de déploiement gère notamment :

- la création et la migration des tables avec SQLAlchemy ;
- la génération des certificats HTTPS avec Let's Encrypt / Certbot.

Dans la configuration Nginx, nous forçons l'utilisation de SSL/HTTPS afin de limiter les attaques de type sniffing, man-in-the-middle ou usurpation de serveur.

5 Interface de l'application

Notre application suit un design assez old school et très numérique avec une police et des widget très carré. Ce choix permet à l'application d'avoir une identité unique et qui correspond au but d'avoir une messagerie chiffré. Lou-Ann a travailler sur le logo, il représente une personne disant shhh.

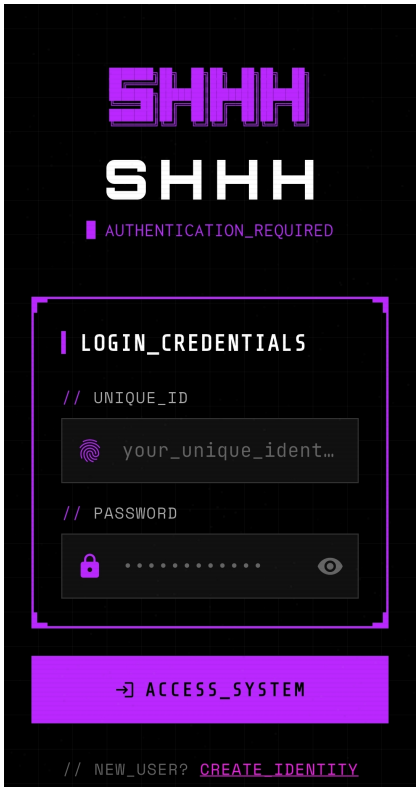


Figure 1: Écran de connexion

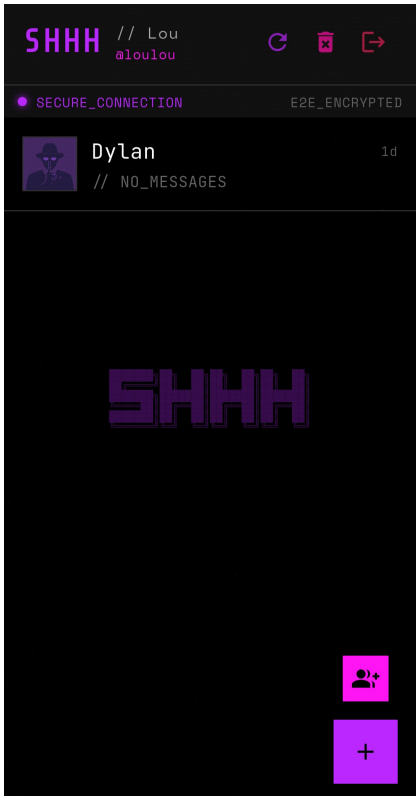


Figure 2: Écran principal

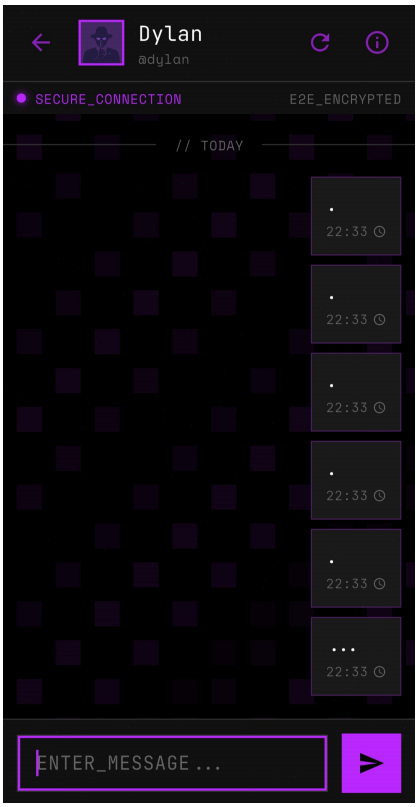
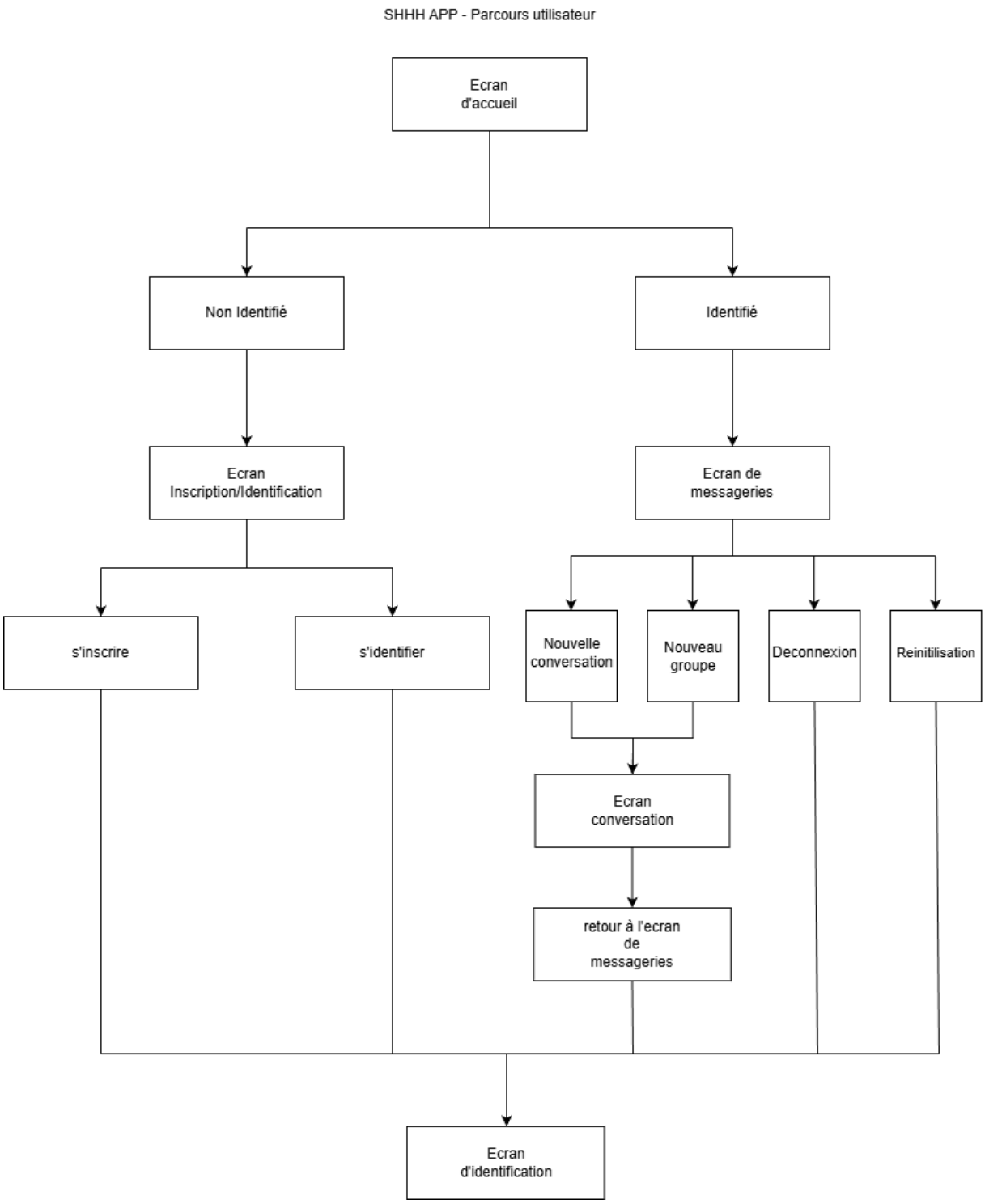


Figure 3: Interface de discussion

6 Parcours utilisateur